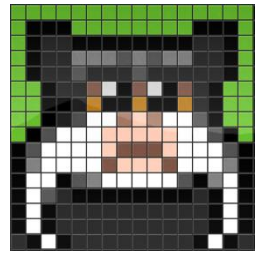
Analyzing Cryptographic Protocols with Tamarin



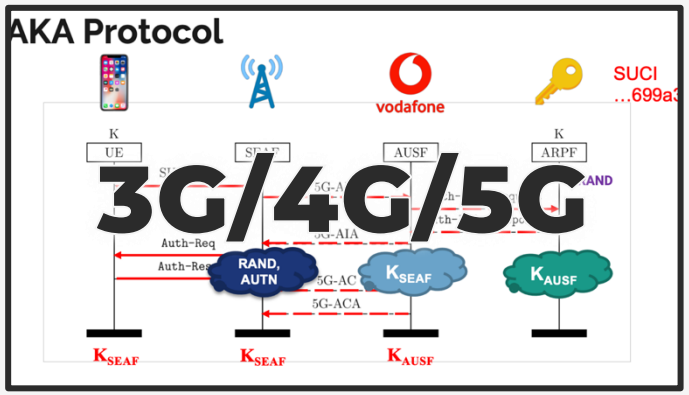
Fifteenth SRI Summer School on Formal Techniques, Menlo Park, 2026

Session B

Cas Cremers

CISPA
HELMHOLTZ CENTER FOR INFORMATION SECURITY

We use security protocols daily



Tamarin prover

Theorem Prover

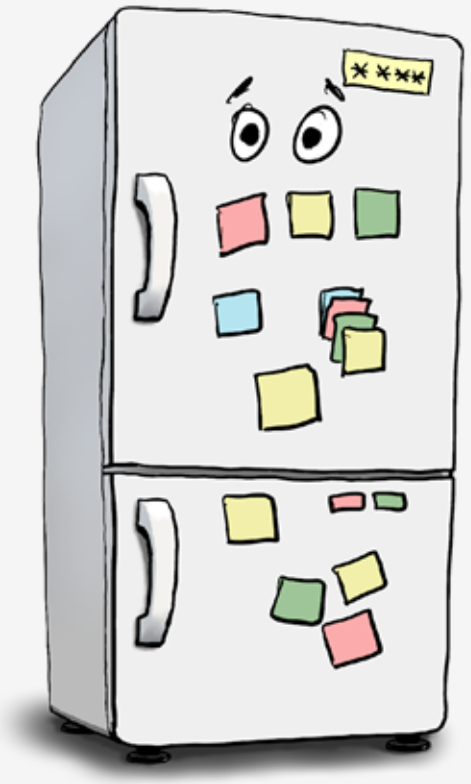
Constraint solver

Modeling in Tamarin

- Basic ingredients:
  - **Terms** (think “messages”)
  - **Facts** (think “sticky notes on the fridge”)
  - Special facts: **Fr(t)**, **In(t)**, **Out(t)**, **K(t)**
- State of system is a multiset of facts
  - **Initial state** is the empty multiset
  - **Rules** specify the transition rules (“moves”)
- Rules are of the form:

  l --> r

  l --[a]-> r



The model

- **Term algebra**
  - enc(_,_), dec(_,_), h(_,_), _^_, _$^{-1}$, _*_, 1, …
- **Equational theory**
  - dec(enc(m,k),k) $=_E$ m,
  - (x^y)^z $=_E$ x^(y*z),
  - $(x^{-1})^{-1} =_E$ x, ...
- **Facts**
  - f(t1,...,tn)
- **Transition system**
  - State: multiset of facts
  - Rules: l –[a]→ r
- **Tamarin-specific**
  - Built-in Dolev-Yao attacker rules:
    - In(t)
    - Out(t)
    - K(t)
  - Special **Fresh** rule:
    - [] --[]--> [Fr(**x**)]
      - Constraint on system such that **x** is unique

Semantics

- **Transition relation**
  - $S \;–[a]\rightarrow_R\; ((\, S \setminus^{\#} I \,) \cup^{\#} r\,)$, where
    - $I \;–[a]\rightarrow\; r$ is a ground instance of a rule in R, and
    - $I \subseteq^{\#} S$ wrt the equational theory
- **Executions**
  - $\mathrm{Exec}(R) =$ $\{ [\,] \;–[a_1]\rightarrow \ldots –[a_n]\rightarrow S_n$ $\mid \forall n \,.\, \mathrm{Fr}(n)$ appears only once on right-hand side of rule $\}$
- **Traces**
  - $\mathrm{Traces}(R) = \{ [a_1,\ldots,a_n] \mid [\,] \;–[a_1]\rightarrow \ldots –[a_n]\rightarrow S_n \in \mathrm{Exec}(R) \}$

Semantics: adversary

- **Adversary-controlled network modeled by Out/In**
  - Adversary learns all terms x from each Out(x)
  - In(x) can be instantiated with any x the adversary can construct

- **Adversary deduction**
  - Adversary can deduce new terms by applying functions and equational theories
    - Concretely: encrypt messages, hash, sign, decrypt when it knows the key, …
- **Extending the adversary**
  - Concrete protocol models can introduce additional adversary capabilities
  - E.g. leaking (compromising) cryptographic keys

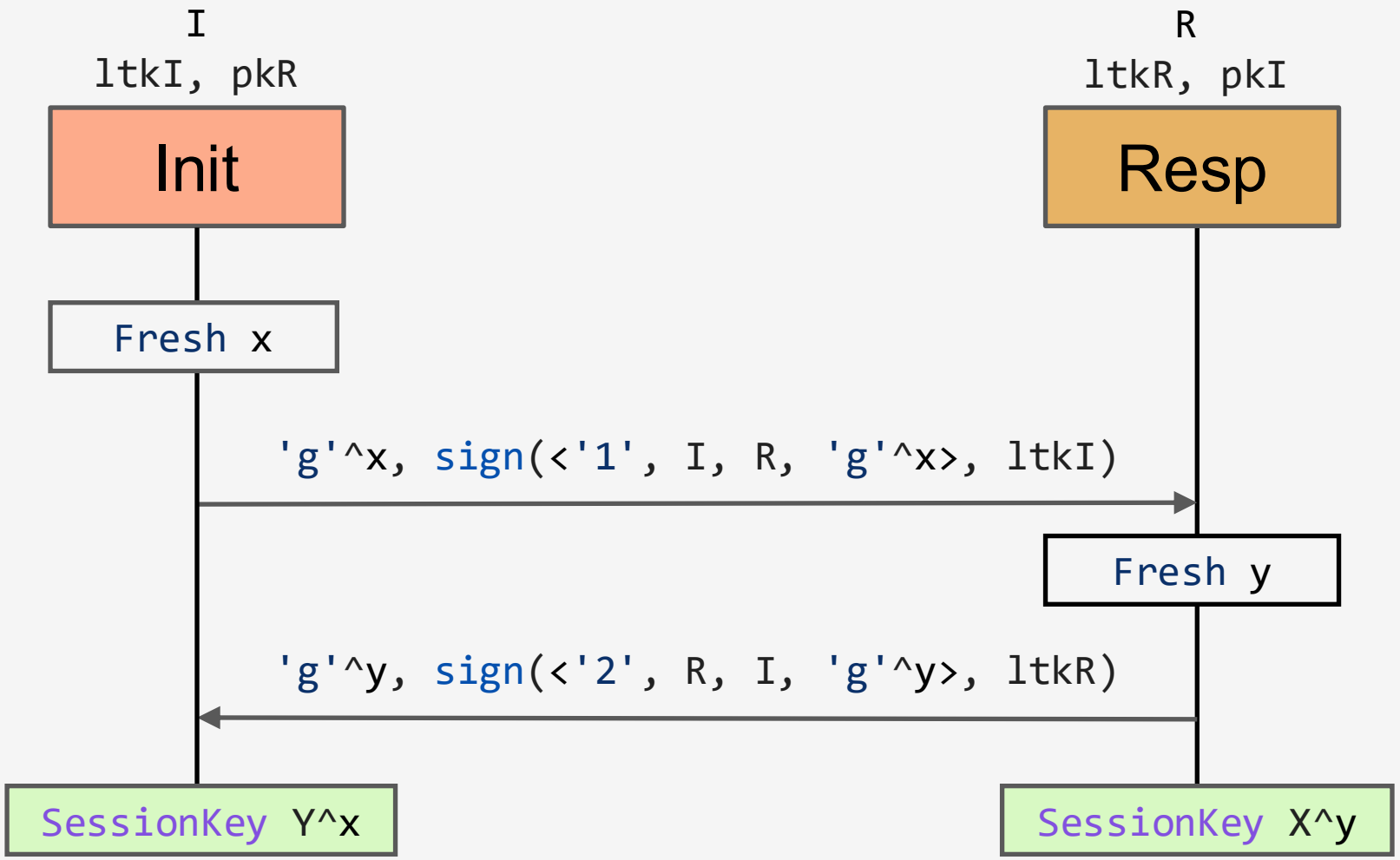
Specifying trace properties

- **Fragment of first-order logic with quantification over timepoints**
  - F(t1...tn) @ #i F(t1…tn) appears at timepoint #i in the trace
  - Action facts on rules and K(t) for adversary-derived terms

  - Logical connectives And, or, implication, negation
  - Equality Over terms, facts, or timepoints
  - Timepoint ordering #i < #j
  - Quantifiers Forall, Exists (All, Ex)

- **Universal and existential quantification over all traces of the system**
  - P(t) holds for all traces t
  - There exists a trace t for which P(t) holds

Signed Diffie-Hellman Protocol



The protocol model

```
theory SignedDH_simple begin
builtins: diffie-hellman, signing

rule Register_pk:
  [ Fr(~ltk) ]
-->
  [ !Ltk($A, ~ltk), !Pk($A, pk(~ltk)), Out(pk(~ltk)) ]

rule Init_1:
  [ Fr(~tid), Fr(~x), !Ltk($I, ltkI) ]
-->
  [ Init_1( ~tid, $I, $R, ~x )
  , Out( <$I, $R, 'g'^~x,
    sign(<'1', $I, $R,'g'^~x>,ltkI) > ) ]

rule Init_2:
  [ Init_1( ~tid, $I, $R, ~x )
  , !Pk($R, pkR)
  , In( <$R, $I, Y, m2> )
  ]
--[ _restrict(verify(m2, <'2', $R, $I, Y>, pkR) = true )
  , SessionKey($I,$R, Y^~x) ]->
  []

rule Resp:
  [ !Pk($I, pkI)
  , !Ltk($R, ltkR)
  , Fr(~y)
  , In( <$I, $R, X, m1> )
  ]
--[ _restrict(verify(m1, <'1', $I, $R, X>,
pkI) = true )
  , SessionKey($I,$R, X^~y) ]->
  [ Out( <$R, $I, 'g'^~y,
    sign(<'2', $R, $I, 'g'^~y >, ltkR)> ) ]


lemma Secrecy:
  "All I R sessKey #i.
    SessionKey(I,R,sessKey)@i
    ==> not (Ex #j. K(sessKey)@j) "

end
```

Topics

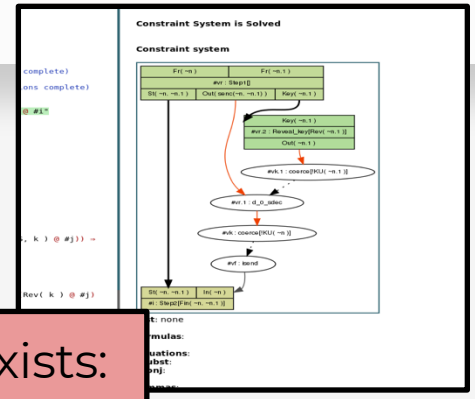
Tamarin's algorithm and modeling features

Threat models

Authentication

The future!

Tamarin prover: workflow



Input file

System S

Property P

Tamarin prover

constraints from S

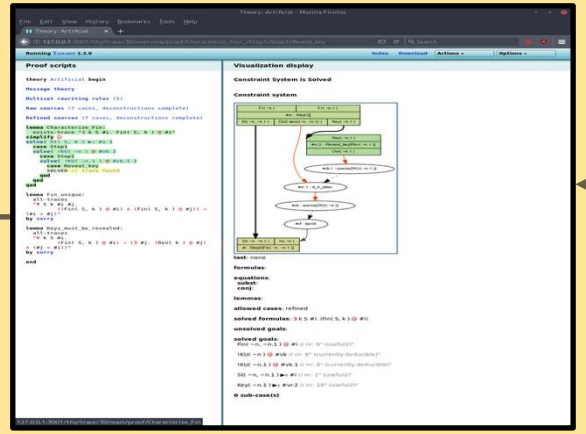
constraints from (not P)

Dedicated constraint solver

Solution exists: ATTACK

No solution exists: PROOF

Run out of time or memory



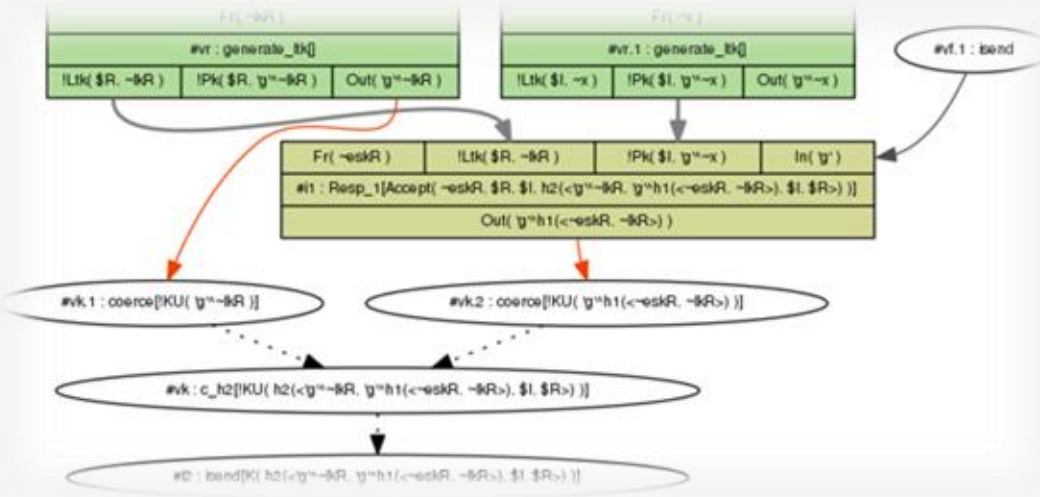
Interactive mode
(Inspect or guide partial proof)

Provide **hints** for the prover (e.g. invariants)

Relevance of counterexamples

1. We are often checking security properties: counterexamples are **attacks**

2. Our models are **abstractions**: counterexamples help us to **catch modeling errors**



Tamarin prover: workflow

Internet Engineering Task Force (IETF) E. Rescorla
Request for Comments: 8446 Mozilla
Obsoletes: 5077, 5246, 6961 August 2018
Updates: 5705, 6066
Category: Standards Track
ISSN: 2070-1721

The Transport Layer Security (TLS) Protocol Version 1.3

Abstract

This document specifies version 1.3 of the Transport Layer Security (TLS) protocol. TLS allows client/server applications to communicate over the Internet in a way that is designed to prevent eavesdropping, tampering, and message forgery.

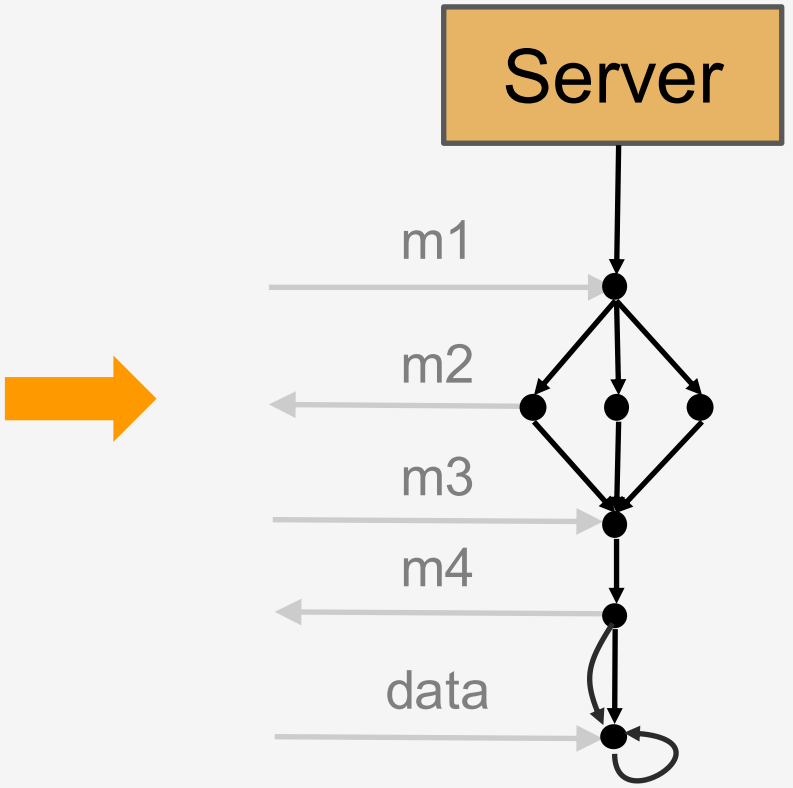
This document updates RFCs 5705 and 6066, and obsoletes RFCs 5077, 5246, and 6961. This document also specifies new requirements for TLS 1.2 implementations.

Status of This Memo

This is an Internet Standards Track document.

This document is a product of the Internet Engineering Task Force (IETF). It represents the consensus of the IETF community. It has received public review and has been approved for publication by the Internet Engineering Steering Group (IESG). Further information on Internet Standards is available in Section 2 of RFC 7841.

Information about the current status of this document, any errata,



How do I know my model is correct?

- **It is easy to model something incorrectly**
- Executability: try to prove expected traces actually exist

How do I know my model is correct?

- **It is easy to model something incorrectly**
- Executability: try to prove expected traces actually exist
- Break the protocol on purpose

Abstraction levels

Reality

Computational

Abstraction levels in theory

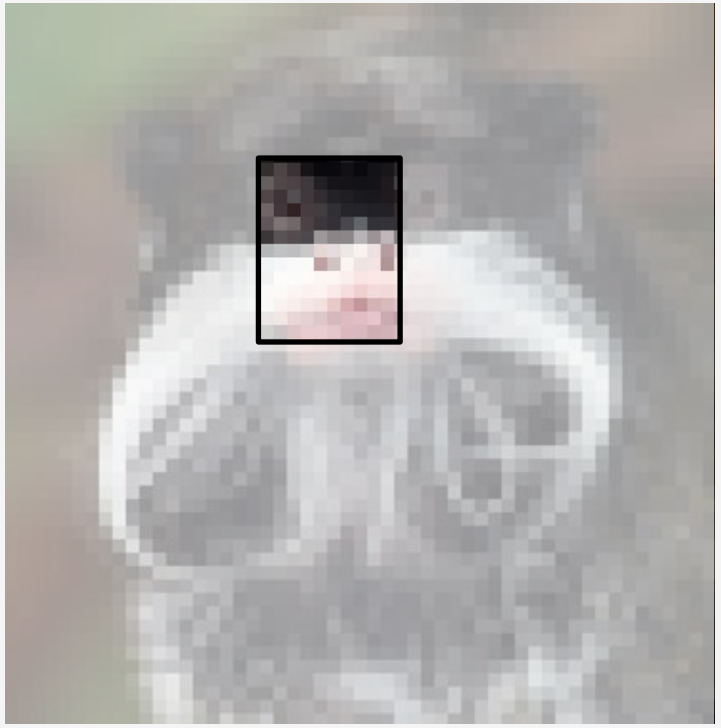
Reality

Computational

Symbolic

Abstraction levels in practice

Reality



Computational

Symbolic

Tamarin's algorithm and modeling features

Basic principles

- Backwards search using constraint reduction (proof) rules
- Turn negation of formula into set of constraints
- Case distinctions
  - E.g.: Possible sources of a message or fact

Basic principles

- Backwards search using constraint reduction (proof) rules
- Turn negation of formula into set of constraints
- Case distinctions
  - E.g.: Possible sources of a message or fact
- Each step of the proof, solve goals:
  - Solve premises based on Dependency Graphs
  - Deconstruction Chains
  - Solve Action goals, …
- If multiple rules can be applied: use heuristics

- **For details, see Chapter 6 of the Tamarin book**

Lemmas

- When Tamarin does not terminate…
- Guide the proof manually; export
- Write **lemmas**
  - "**Hints**" for the prover
    - They don't change the guarantees, only help tool in finding a proof
  - E.g. specify lemma that can be used to prune proof trees at multiple points

Motivating example

- Consider the following model:

```
theory Loop
begin

rule start:  [ Fr(x) ] --[ Start(x) ]-> [ A(x) ]
rule repeat: [  A(x) ] --[  Loop(x) ]-> [ A(x) ]
rule stop:   [  A(x) ] --[  Stop(x) ]-> [      ]

lemma AlwaysStarts:
  " All x #i . Loop(x)@i ==> Ex #j. Start(x)@j "

lemma AlwaysStartsWhenEnds:
  " All x #i . Stop(x)@i ==> Ex #j. Start(x)@j "

end
```

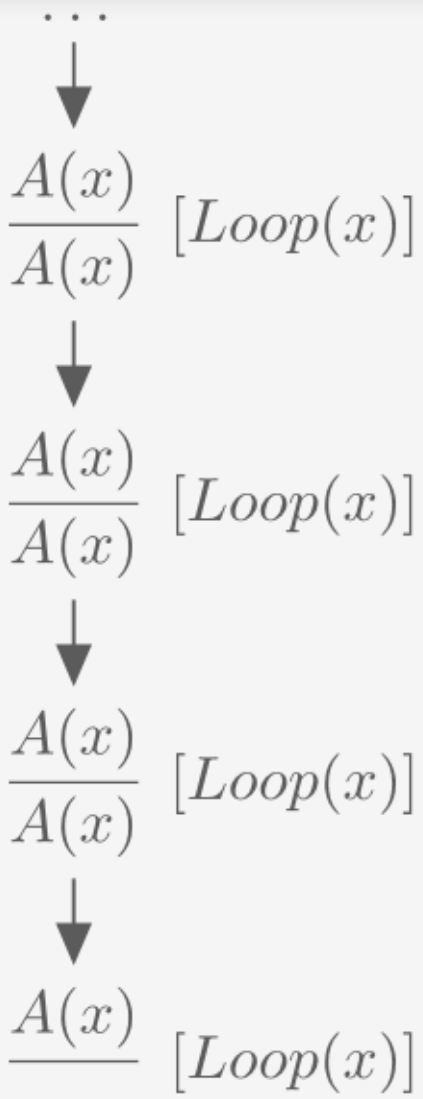
- If you try to verify either lemma with Tamarin's default heuristic, it will cause an infinite loop. Why?

Reasoning about loops

- Our protocol consists of three rules:

```
rule start:
  [ Fr(x) ]–[ Start(x) ]->[ A(x) ]

rule repeat:
  [ A(x) ]--[ Loop(x) ]->[ A(x) ]

rule stop:
  [ A(x) ]--[ Stop(x) ]->[ ]
```

- How would Tamarin reason backwards from the action fact `Loop(X)` to find a counterexample?



Induction

- Annotation: `[use_induction]`

```
lemma AlwaysStarts [use_induction]:
    " All x #i . Loop(x)@i ==> Ex #j. Start(x)@j "
```

- Can also be chosen from the GUI

Reuse

- Annotation: `[reuse]`

```
lemma AlwaysStarts [use_induction, reuse]:
    " All x #i . Loop(x)@i ==> Ex #j. Start(x)@j "
```

- Tells Tamarin to assume that the lemma holds when proving subsequent lemmas
- You need to make sure that the lemma proves before reusing it
  - Tamarin will use it regardless, but the results will be wrong
- Does not always reduce proof time
  - Use with caution; do not mark everything reusable

Solving the loop problem

- In the loop example, we first prove that a `Loop(x)` action is always preceded by a `Start(x)` action
- Then, we reuse the assumption when proving that a `Stop(x)` action is always preceded by a `Start(x)` action

```
lemma AlwaysStarts [use_induction, reuse]:
  " All x #i . Loop(x)@i ==> Ex #j. Start(x)@j "

lemma AlwaysStartsWhenEnds [use_induction]:
  " All x #i . Stop(x)@i ==> Ex #j. Start(x)@j "
```

- Including the `[reuse]` annotation allows Tamarin to assume that the lemma holds for all following lemmas

Restrictions

- Syntactically similar to lemmas, but used to exclude traces
  - Unlike lemmas, you do not need to prove restrictions
- For example, we can define a restriction `Eq(x,y)`, s.t., whenever it is used, Tamarin will skip any traces where x and y are not equal

```
restriction Equality:
    " All x y #i. Eq(x,y)@i ==> x = y "
```

- This can be used instead of pattern matching for e.g., ensuring that two signatures are equal

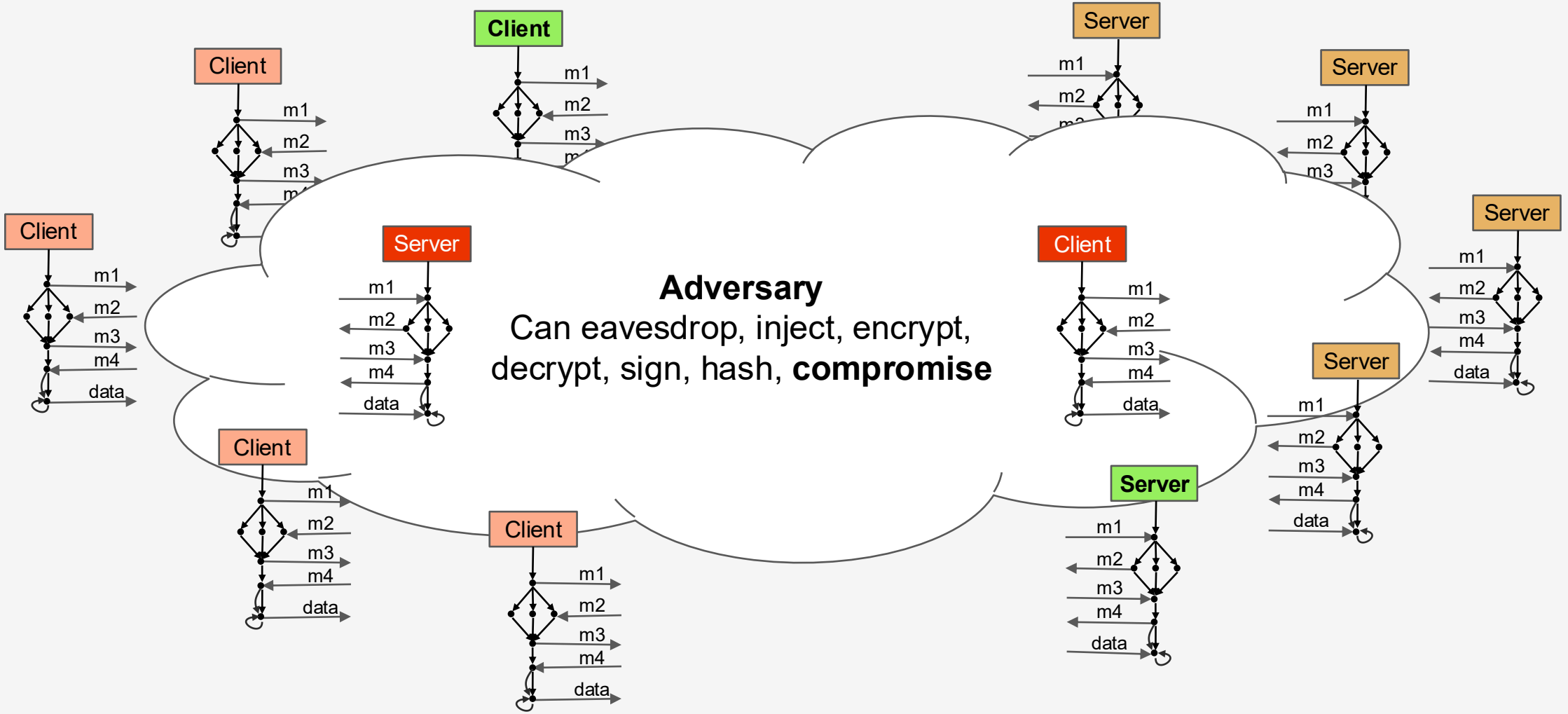
```
rule restriction_example:
    [ In(SignA), In(SignB) ] --[ Eq(SignA, SignB) ]-> [ ]
```

Commonly used restrictions

```
restriction Unique:
    " All x #i #j. Unique(x)@i & Unique(x)@j ==> #i = #j "

restriction OnlyOnce:
    " All #i #j. OnlyOnce()@i & OnlyOnce()@j ==> #i = #j "

restriction Equality:
    " All x y #i. Eq(x,y)@i ==> x = y "

restriction Inequality:
    " All x #i. Neq(x,x)@i ==> F "
```

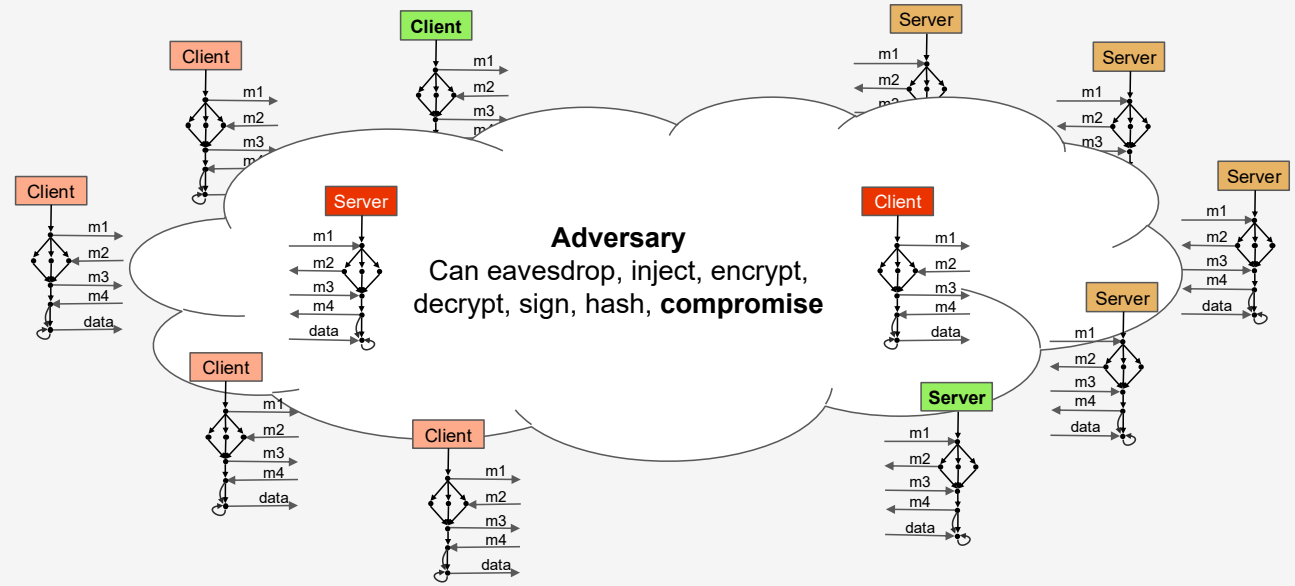
Threat models

Security protocols and threat models



Threat model

- Threat model: Adversary capabilities
- So far, we have considered the default Dolev-Yao adversary model
  - All messages are sent to the attacker
  - Attacker can use keys to sign, decrypt, or encrypt messages, hash, combine, etc
- In practice, we often want to consider a stronger or weaker adversary model



Choosing a threat model

- Deciding on a realistic or “correct” threat model is not always easy
- If not specified, try to determine the weakest attacker required to break the system
  - If you find attacks: Weaken the attacker
  - If you cannot find attacks: Strengthen the attacker

- Note: you can automate this sometimes!
  (see e.g., ”A spectral analysis of Noise”, Girol et al, USENIX 2020)

Basic secrecy

- Basic secrecy property: annotate all rules where a secret is computed with an action fact.
- For example: `Secret(A, B, k)` to denote that party `A` computed a key `k`, intended to be shared with `B` :

```
lemma session_key_secrecy:
    " All A B k #i . Secret(A, B, k)@i
        ==> not (Ex #r . K(k)@r)
```

Modeling compromised values

- We strengthen the attacker by adding rules that reveal secrets
- Attacker that can learn session keys (e.g. by cryptanalysis):

```
rule reveal_session_key:
    [ !SessionKey(k) ] --[ RevealSessionKey(k) ]-> [ Out(k) ]
```

- In combination with property specifications, this allows us to reason about the impact of compromise, also known as the "**blast radius**"
- examples:
  - Session-key independence
    (Backward/forward secrecy of session keys)
  - Resilience against compromised randomness/RNG

Example: session key independence

```
rule reveal_session_key:
    [ !SessionKey(k) ] --[ RevealSessionKey(k) ]-> [ Out(k) ]
```

- Note: Attackers that can reveal **all** keys are hard to defend against

- **Session-key independence**:
  "can the attacker learn session key `k` if it learns any *other* session key?"

```
lemma session_key_secrecy_with_key_independence:
    " All A B k #i . Secret(A, B, k)@i
        ==> not (Ex #r . K(k)@r)
            | (Ex #r . RevealSessionKey(k)@r) "
```

Compromised agents

- Protocols often assume that the sender and receiver know each other's public keys before the protocol starts:
  Public-Key Infrastructure (PKI)
- We model it **abstractly** as an initialization rule
- We can then add a **reveal rule for the private key**, after which the attacker can impersonate the party

```
functions: pk/1

/* Create a key pair */
rule register_pk:
    let
        pubkey = pk(~ltk)
    in
    [ Fr(~ltk) ]
  --[ Register($A, ~ltk) ]->
    [ !Ltk($A, ~ltk)
    , !Pk($A, pubkey)
    , Out(pubkey) ]

/* Reveal the long-term key */
rule reveal_ltk:
    [ !Ltk(A, ltk) ]
  --[ RevealLtk(A) ]->
    [ Out(ltk) ]
```

Modeling compromised/dishonest agents

- We consider statements like:
  - “can the attacker compromise a session between A and B if he learns the long term keys of other parties?”
- In lemmas: “the property holds *unless* the adversary compromised some specific parties”

```
lemma session_key_secrecy:
    " All A B k #i . Secret(A, B, k)@i
        ==> not (Ex #r . K(k)@r)
            | (Ex #r . RevealLtk(A)@r)
            | (Ex #r . RevealLtk(B)@r) "
```

- Examples:
  - Secrecy with dishonest insiders
  - Perfect Forward Secrecy
  - Key Compromise Impersonation

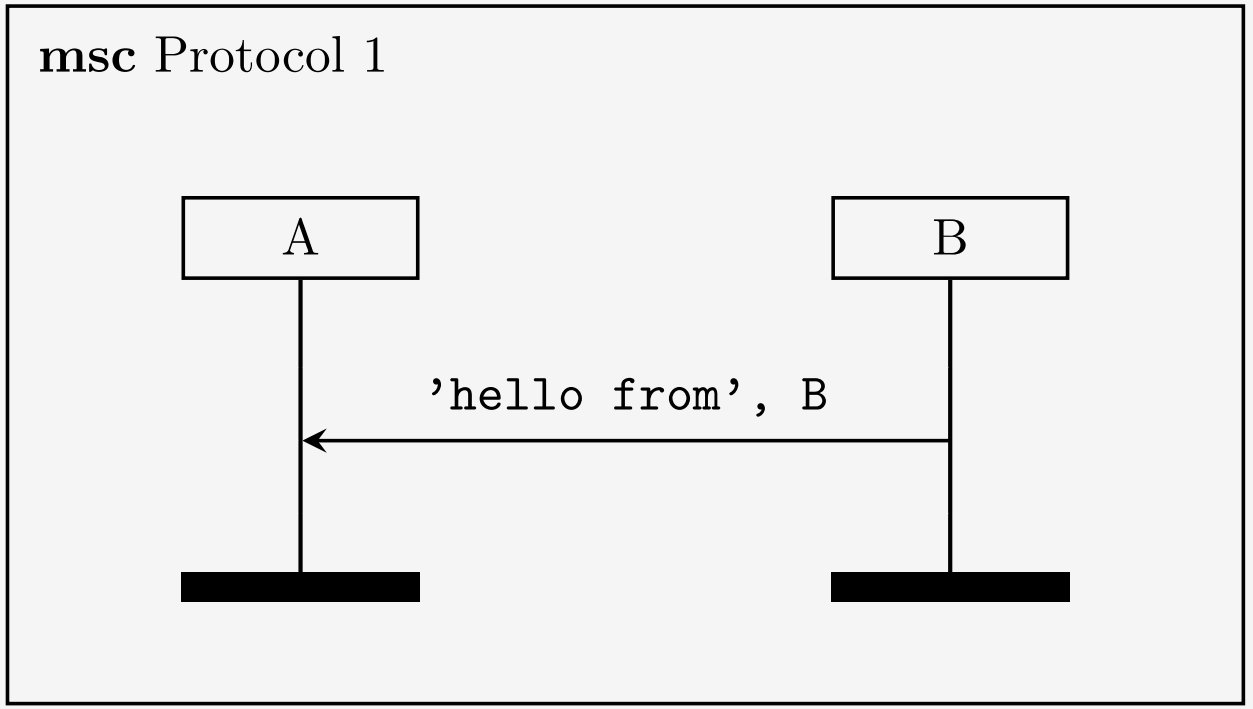
Authentication

Authentication

- **Authentication** also depends on **perspective** (of role)
  - **Unilateral authentication**: one party authenticates the other
  - **Mutual authentication**: both parties authenticate each other

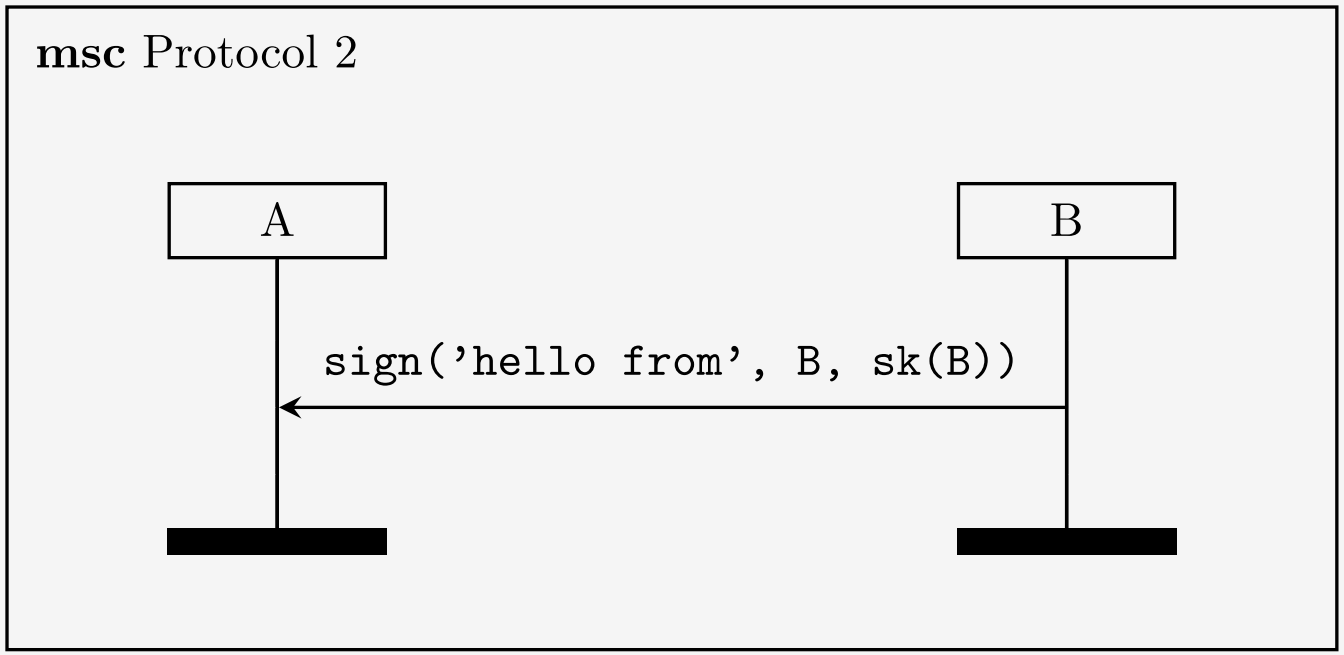
- Here we will only consider **unilateral authentication**

Authentication of B by A



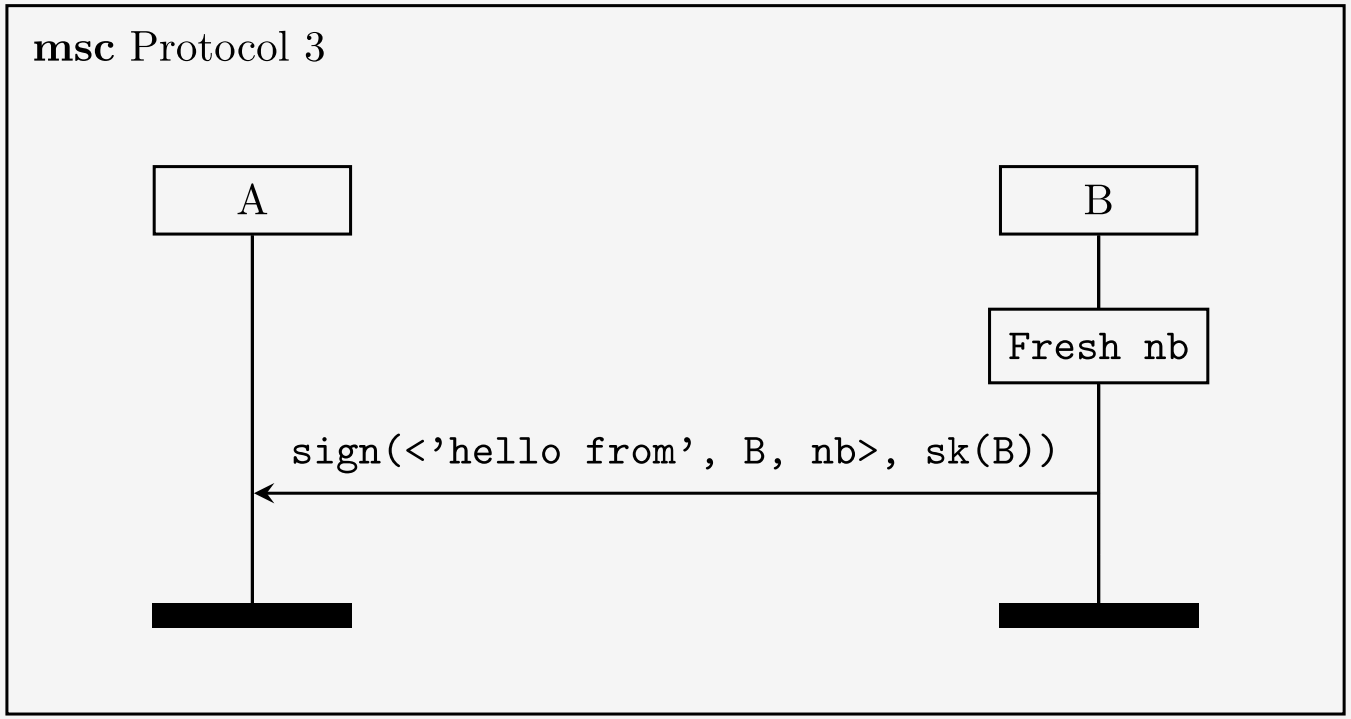
Authentication of B by A

- A network attacker could have just forged the message in Protocol 1.
- Next try:



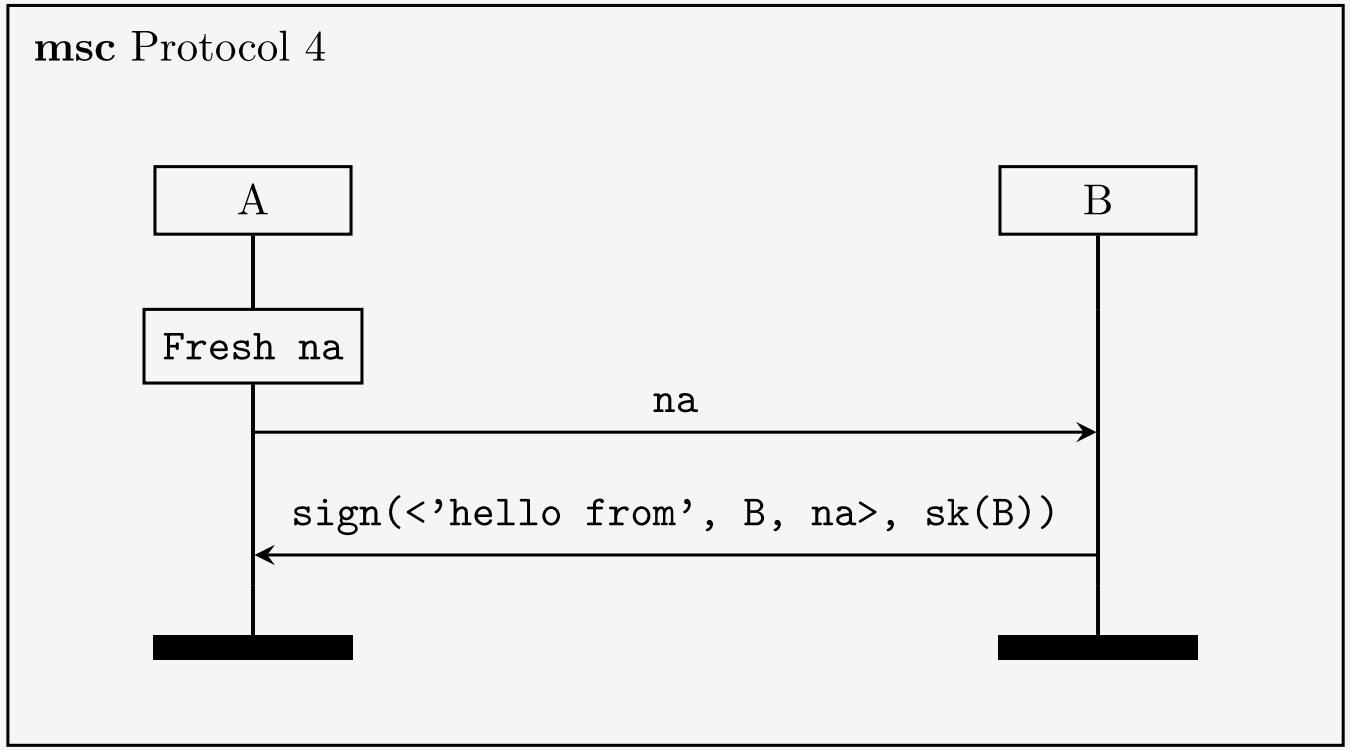
Authentication of B by A

- In Protocol 2, adversary could just replay messages from old sessions.
- Next try: add unique values to each message



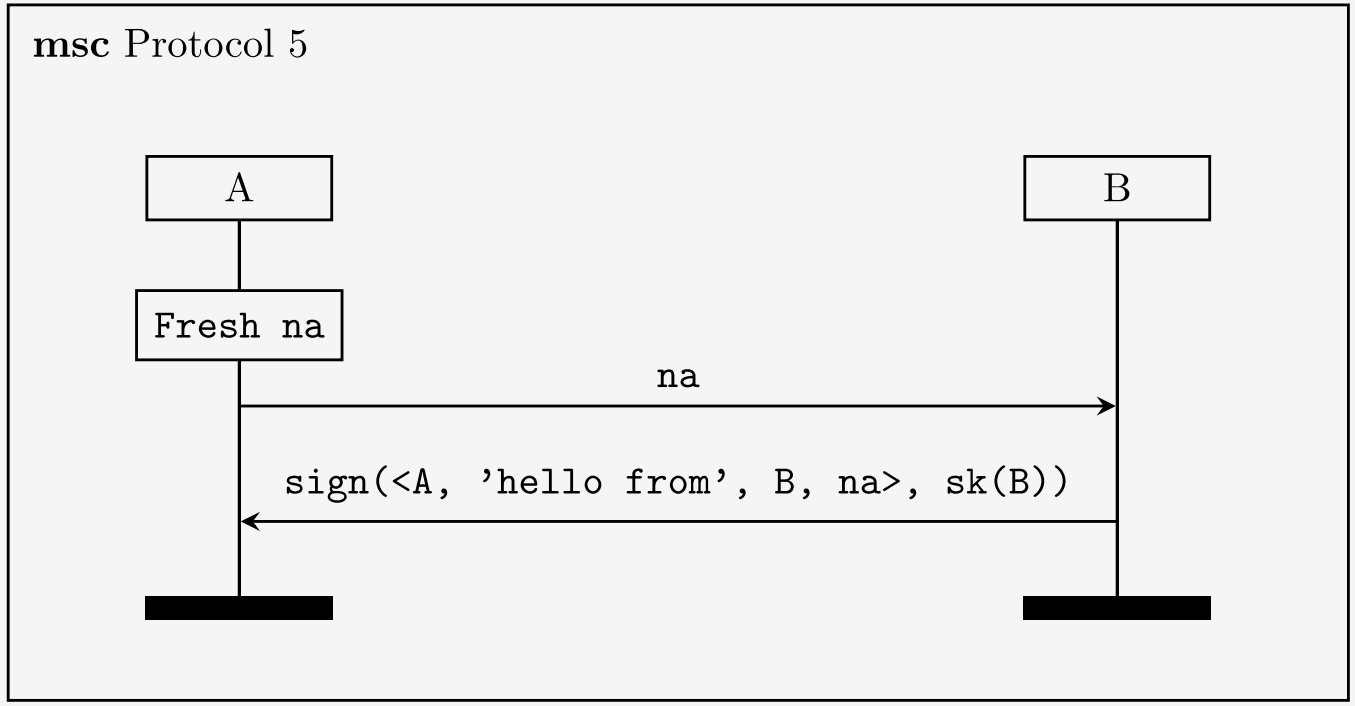
Authentication of B by A

- In Protocol 3, adversary could still replay messages, unless we store all previously seen values.
- Challenge-response is a more reliable pattern that also gives us a temporal guarantee.



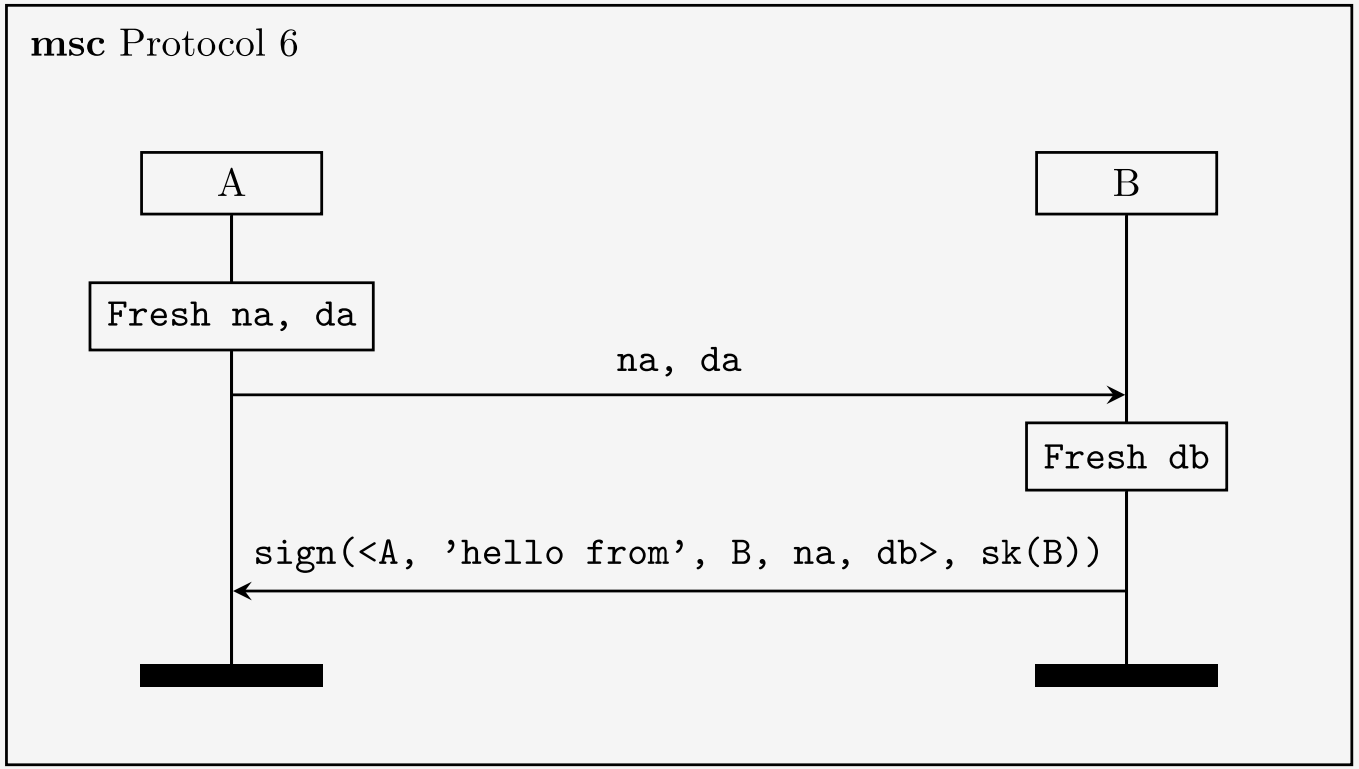
Authentication of B by A

- In Protocol 4, A could not be sure B wanted to talk to them. Perhaps B thought they were responding to C or D.
- We also add A's identity.



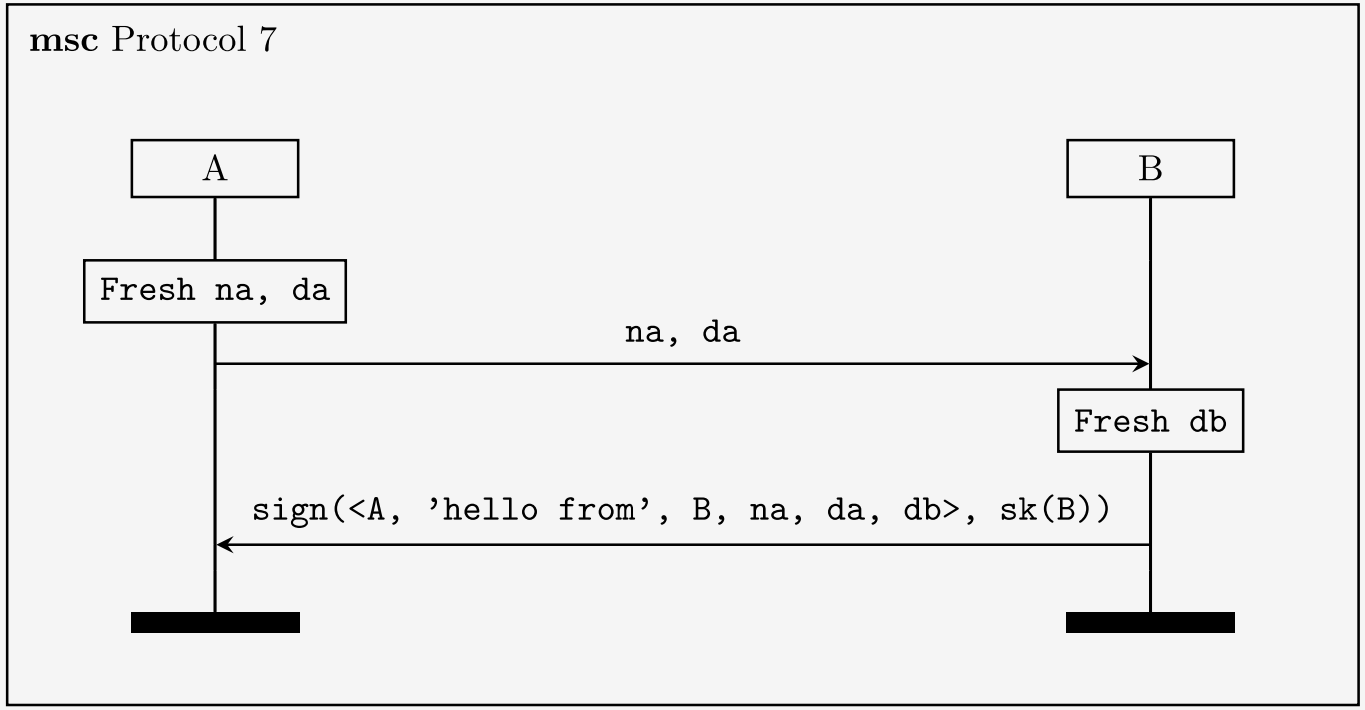
Authentication of B by A

- Let's add some additional data.



Authentication of B by A

- In Protocol 6, A could not be sure whether it agrees with B on da.
- Thus we also include the data in the signature.



Lowe's hierarchy

- In the literature, you will see multiple variations of authentication claims with subtle differences
- In 1997, Gavin Lowe proposed a hierarchy of authentication properties in increasing strength:
    1. aliveness,
    2. weak agreement,
    3. non-injective agreement, and
    4. injective agreement
- Each property applies from either party's perspective

Authentication claims

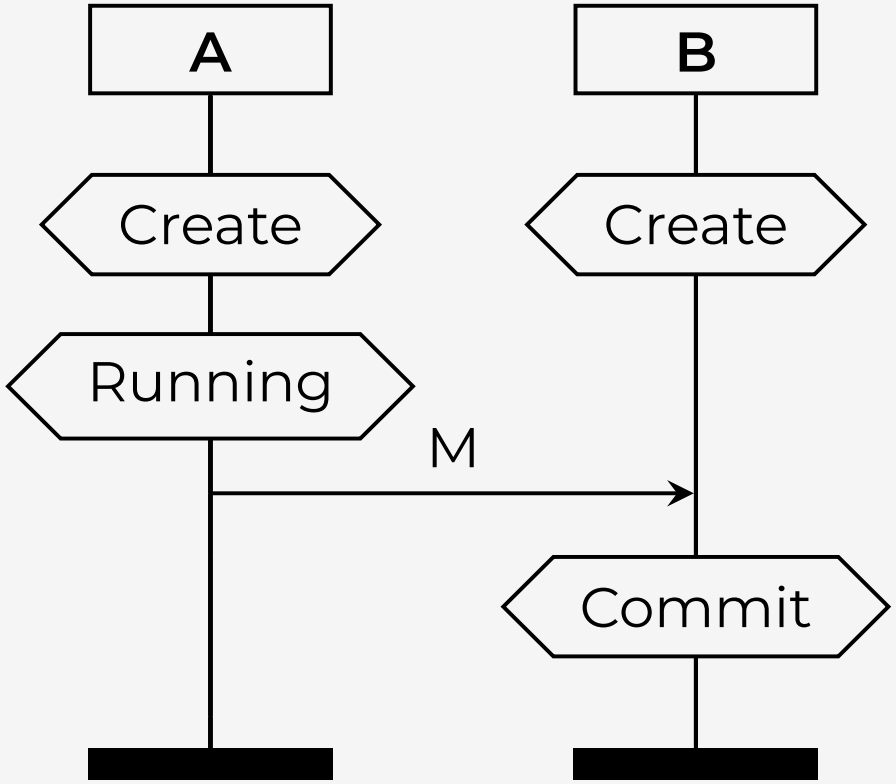
- We typically use the following action facts:

- `Create(X)`
  Create Agent X

- `Running(X,Y,t)`
  Agent X believes to be executing the protocol with agent Y and has stored term t

- `Commit(X,Y,t)`
  Agent X believes to have completed the protocol with agent Y and has stored term t

- `Reveal(X)`
  Agent X has revealed its long-term secret(s)



Aliveness

A protocol guarantees **aliveness**
to an agent *a* in role *A* if, whenever *a* completes a run of the protocol, seemingly with *b* in role *B*, **then *b* has previously been running the protocol.**

```
/* Aliveness from A's perspective */
lemma aliveness:
    " All A B t #i .
        Commit(A, B, t)@i
        & not (Ex #r. Reveal(A)@r)
        & not (Ex #r. Reveal(B)@r)
            ==> (Ex #j. Create(B)@j & j < i)
    "
```

Weak agreement

A protocol guarantees **weak agreement**
to an agent *a* in role *A* if, whenever *a* completes a run of the protocol, seemingly with *b* in role *B*, then *b* has previously been running the protocol, **seemingly with *a*.**

```
/* Weak agreement from A's perspective */
lemma weak_agreement:
    " All A B t #i .
        Commit(A, B, t)@i
        & not (Ex #r. Reveal(A)@r)
        & not (Ex #r. Reveal(B)@r)
            ==> (Ex t2 #j. Running(B, A, t2)@j & j < i)
    "
```

Non-injective agreement

A protocol guarantees non-injective agreement
to an agent *a* in role *A* if, whenever *a* completes a run of the protocol, seemingly with *b* in role *B*, then *b* has previously been running the protocol in role *B*, seemingly with *a*, **and they both agree on the term *t*.**

```
/* Non-injective agreement from A's perspective */
lemma noninjective_agreement:
    " All A B t #i .
        Commit(A, B, t) @i
        & not (Ex #r. Reveal(A)@r)
        & not (Ex #r. Reveal(B)@r)
            ==> (Ex #j. Running(B, A, t)@j & j < i)
    "
```

Injective agreement

A protocol guarantees **injective agreement**
to an agent *a* in role *A* if, whenever *a* completes a run of the protocol, seemingly with *b* in role *B*, then *b* has previously been running the protocol in role *B*, seemingly with *a*, and they both agree on the term *t*. **Each run of agent *a* in role *A* corresponds to a unique run of agent *b*.**

```
/* Injective agreement from A's perspective */
lemma injective_agreement:
    " All A B t #i.
        Commit(A, B, t)@i
        & not (Ex #r. Reveal(A)@r)
        & not (Ex #r. Reveal(B)@r)
            ==> (Ex #j. Running(B, A, t)@j & j < i
                 & not(Ex A2 B2 #i2 . Commit(A2, B2, t)@i2
                    & not(#i = #i2)))
    "
```

The future

Success stories

[TLS] Revision 10: possible attack if client authentication is allowed during PSK

Sam Scott <sam.scott89@gmail.com> | Sat, 31 October 2015 11:19 UTC | Show header

Dear all,

We [1] are in the process of performing an automated symbolic analysis
of the TLS 1.3 specification draft (revision 10) using the Tamarin
prover [2],

While revis
authenticat
what we bel
authenticat
signature over the handshake hash, our Tamarin analysis finds an attack. The

TLS 1.3

Security Research

Blog

iMessage PQ3

October 27, 2023

Advancing iMessage security: iMessage Contact Key Verification

ZDNet

Academics bypass PINs for Visa contactless payments

Researchers: "In other words, the PIN is useless in Visa contactless transactions."

EMV (Chip and Pin)

CVE-2023-31127

SPDM (PCI Express)

OpenCVE > Vulnerabilities (CVE) > CVE-2023-31127

libspdm is a sample implementation that follows the DMTF SPDM specificat
in libspdm prior to version 2.3.1. If a device supports both DHE session and

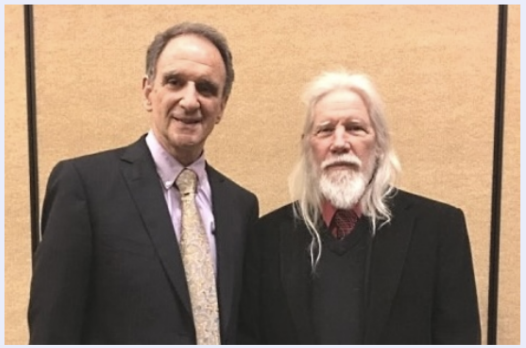
CVSS v3 8.8 HIGH

8.8 /10

IACR 2026 Levchin Prize for Real-World Cryptography

LEVCHIN PRIZE WINNERS

2026



MARTIN HELLMAN AND WHIT DIFFIE

Citation

For the invention of public key cryptography.

Read more ...

DAVID BASIN, CAS CREMERS, JANNIK DREIER, RALF SASSE

Citation

For developing the Tamarin prover, and its use in the analysis of real-world security protocols.

Read more ...

Many challenges remain

Tamarin is very far from done!

- Modeling real-world protocols (understanding, abstraction)
- Scaling: Verification of the most complex models
- Proof robustness under protocol changes

The future

Formal analysis advances
new abstraction levels
more practical compositional reasoning

New designs will come with formal analysis
either *mandatory* or as *competitive advantage*

Quantum computers?
redesign of protocols & infrastructure

Geopolitical landscape

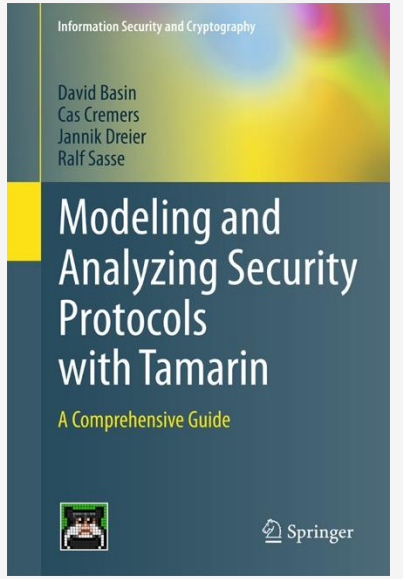
The interaction between

- the need to globally interoperate and
- sovereignty (either at the national or regional level),

will require **methodology** to **objectively assess and certify** the security guarantees of systems and protocols.

Conclusions

- tamarin-prover.com
- Open source, well documented, and ready for use
- In preparation: Tamarin foundation
- Reach out to us or the Tamarin mailing list



Cas Cremers (cremers@cispa.de)